

noun.wtf

Hackathon Guide

Everything you need to build on the noun.wtf stack: contracts, APIs, indexer, autonomous agent. Built for Nouns, client ID 37.

Table of Contents

1. [What Is This](#)
 2. [The Nouns Ecosystem \(30 Second Version\)](#)
 3. [Architecture](#)
 4. [Contracts On Chain](#)
 5. [GraphQL API](#)
 6. [REST Endpoints](#)
 7. [Agent NounIRL](#)
 8. [Small Grants Treasury](#)
 9. [Terminal Chat \(Claude AI + Governance Tools\)](#)
 10. [Local Development](#)
 11. [Environment Variables](#)
 12. [Key Integration Points](#)
 13. [Build Ideas](#)
-

What Is This

noun.wtf is a governance hub for Nouns DAO. It indexes all onchain activity (auctions, proposals, votes, delegations, streams, grants), serves it through a GraphQL API, and wraps it in a terminal-style UI with an AI agent that can execute governance actions.

The stack is:

- **Frontend** — Vite 6 + React 19 + wagmi v2 + viem
- **Indexer** — Ponder v0.16 (Postgres-backed, GraphQL out of the box)

- **Agent** — Autonomous Noun settler + Claude AI terminal
- **Contracts** — SmallGrantsTreasury (noun.wtf-exclusive micro-governance)

Everything runs on Ethereum mainnet. Client ID [37](#).

The Nouns Ecosystem

One Noun is auctioned every 24 hours, forever. The proceeds go to the Nouns DAO treasury (~\$50M+). Noun holders (1 NFT = 1 vote) govern the treasury through onchain proposals.

Key primitives:

Concept	What It Means
Noun	ERC-721 NFT with procedurally generated pixel art (5 traits: background, body, accessory, head, glasses)
Auction	24hr English auction for each new Noun. Winner gets the NFT + 1 governance vote
Proposal	Onchain executable code (send ETH, call contracts) requiring majority vote to pass
Candidate	Offchain proposal draft that needs sponsor signatures before going onchain
Delegation	Noun holders can delegate their votes to another address
Stream	Continuous payment stream from treasury to a recipient over time
Fork	Minority protection — Noun holders can ragequit with their pro-rata treasury share
Client Incentive	Protocol rewards for frontends that facilitate votes/bids (identified by client ID)

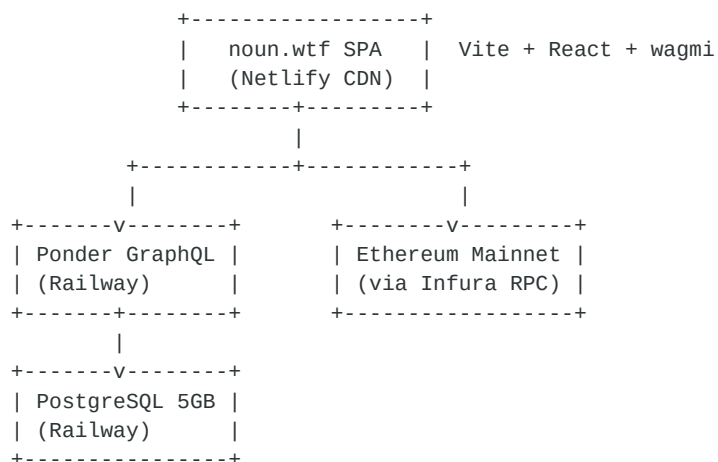
Contract addresses that matter:

```

NounsToken:          0x9C8fF314C9Bc7F6e59A9d9225Fb22946427eDC03
NounsAuctionHouseV2: 0x830BD73E4184ceF73443C1511a1DF14e495C706
NounsDAOV4:          0x6f3E6272A167e8AcCb32072d08E0957F9c79223d
NounsDAOData:        0xf790A5f59678dd733fb3De93493A91f472ca1365
StreamFactory:        0x0fd206FC7A7dBcD5661157eDCb1FFDD0D02A61ff
SmallGrantsTreasury: 0xBAC9233725440c595b19d975309CC98cb259253a

```

Architecture



The frontend is a pure client-side SPA. It talks to:

1. **Ponder API** — for indexed historical data (proposals, votes, auctions, etc.)
2. **Ethereum RPC** — for live reads + write transactions (via wagmi/viem in the browser)

Ponder watches the chain via Infura RPC, indexes events into Postgres, and exposes a GraphQL API automatically from the schema definition.

Contracts On Chain

What's Indexed

Contract	Address	Start Block	Events Indexed
NounsAuctionHouseV2	0x830BD73E...	12,985,451	AuctionCreated, AuctionBid, AuctionSettled, AuctionBidWithClientId
NounsToken	0x9C8fF314...	12,985,438	NounCreated, Transfer, DelegateChanged, DelegateVotesChanged

Contract	Address	Start Block	Events Indexed
NounsDAOV4	0x6f3E6272...	12,985,453	ProposalCreated, VoteCast, VoteCastWithClientId, ProposalQueued, ProposalExecuted, ProposalCanceled, ProposalVetoed, ProposalObjectionPeriodSet, ProposalUpdated
NounsDAOData	0xf790A5f5...	17,812,145	ProposalCandidateCreated, ProposalCandidateUpdated, ProposalCandidateCanceled, SignatureAdded, FeedbackSent, CandidateFeedbackSent
StreamFactory	0x0fd206FC...	16,576,500	StreamCreated
Stream	(factory children)	16,576,500	TokensWithdrawn, StreamCancelled, StreamConcluded
SmallGrantsTreasury	0xBAc92337...	24,650,190	ProposalCreated, VoteCast, ProposalQueued, ProposalExecuted, ProposalCanceled

Sepolia Testnet

Same contracts exist on Sepolia for testing:

NounsToken: 0x4C4674bb72a096855496a7204962297bd7e12b85
 NounsAuctionHouseV2: 0x488609b7113FCf3B761A05956300d605E8f6BcAf
 NounsDAOV4: 0x35d2670d7C8931AACdd37C89Ddcb0638c3c44A57
 NounsDAOData: 0x9040f720AA8A693f950b9cf94764b4b06079D002
 StreamFactory: 0xb78ccF3BD015f209fb9B2d3d132FD8784Df78DF5

GraphQL API

Endpoint: <https://spirited-flexibility-production-3c30.up.railway.app/graphql>

POST a JSON body with `{ "query": "..."}` . The API supports the full Ponder GraphQL schema with filtering, ordering, and pagination.

Schema Overview

nouns — Every Noun ever minted

```
{
  nouns(orderBy: "id", orderDirection: "desc", limit: 10) {
    items {
      id          # bigint - Noun ID
      owner       # hex - current owner
      head        # int - trait index
      body        # int - trait index
      accessory   # int - trait index
      glasses     # int - trait index
      background  # int - 0 or 1
      createdAt
      createdAtBlock
    }
  }
}
```

proposals — DAO governance proposals

```
{
  proposals(orderBy: "id", orderDirection: "desc", limit: 20) {
    items {
      id
      proposer
      description      # full markdown text
      status           # PENDING | ACTIVE | CANCELLED | VETOED | QUEUED | EXECUTED
      forVotes
      againstVotes
      abstainVotes
      startBlock
      endBlock
      executionETA
      objectionPeriodEndBlock
      updatePeriodEndBlock
      voteSnapshotBlock
      clientId         # 37 = noun.wtf
      createdAt
    }
  }
}
```

votes — Individual vote records

```
{
  votes(where: { proposalId: "123" }, orderBy: "createdAtBlock", orderDirection: "desc") {
    items {
      voter
      proposalId
      support      # 0=AGAINST, 1=FOR, 2=ABSTAIN
      votes        # voting power (# of Nouns)
      reason       # vote reason text
      clientId
      createdAt
    }
  }
}
```

auctions — Auction history

```
{
  auctions(orderBy: "nounId", orderDirection: "desc", limit: 10) {
    items {
      nounId
      startTime
      endTime
      settled
      winner
      amount          # winning bid in wei
      clientId
    }
  }
}
```

bids — All bids on auctions

```
{
  bids(where: { nounId: "1234" }, orderBy: "value", orderDirection: "desc") {
    items {
      nounId
      bidder
      value          # bid amount in wei
      clientId
    }
  }
}
```

delegates — Delegation power

```
{
  delegates(orderBy: "delegatedVotes", orderDirection: "desc", limit: 50) {
    items {
      id              # address
      delegatedVotes  # total voting power
    }
  }
}
```

streams — Payment streams from treasury

```
{
  streams(orderBy: "startTime", orderDirection: "desc") {
    items {
      streamAddress
      proposalId
      status          # active | cancelled | concluded
      creator
      payer
      recipient
      tokenAmount
      withdrawnAmount
      tokenAddress
      startTime
      stopTime
    }
  }
}
```

candidates — Offchain proposal drafts

```
{
  candidates(orderBy: "createdAt", orderDirection: "desc", limit: 20) {
    items {
      id          # "{proposer}-{slug}"
      slug
      proposer
      canceled
      versionsCount
      description
      createdAt
      lastUpdatedAt
    }
  }
}
```

grants — SmallGrantsTreasury proposals (noun.wtf exclusive)

```
{
  grants(orderBy: "id", orderDirection: "desc") {
    items {
      id
      proposer
      description
      status          # ACTIVE | DEFEATED | SUCCEEDED | QUEUED | EXECUTED | CANCELED | EXPIRED
      forVotes
      againstVotes
      abstainVotes
      snapshotBlock
      startBlock
      endBlock
      executionETA
      createdAt
    }
  }
}
```

grantVotes — Votes on grants

```
{
  grantVotes(where: { grantId: "1" }) {
    items {
      voter
      grantId
      support
      votes
      reason
    }
  }
}
```

Filtering

Ponder GraphQL supports **where** with comparison operators:

```
{
  votes(where: { voter: "0x1234...", support: 1 }) {
    items { proposalId votes reason }
  }
}
```

```
}  
}
```

Pagination

Use **limit** and **after** cursor:

```
{  
  proposals(limit: 10, after: "cursor_string") {  
    items { id status }  
    pageInfo { hasNextPage endCursor }  
  }  
}
```

REST Endpoints

Base URL: <https://spirited-flexibility-production-3c30.up.railway.app>

Activity Feed

GET /api/activity?type=VOTE&limit=50&before=12345678

Returns recent onchain events. Supported types: **VOTE**, **AUCTION_BID**, **AUCTION_SETTLED**, **PROPOSAL_CREATED**, **PROPOSAL_STATUS_CHANGE**, **DELEGATION**, **TRANSFER**, **STREAM_CREATED**, **CANDIDATE_CREATED**, **CANDIDATE_UPDATED**, **CANDIDATE_CANCELED**, **SIGNATURE_ADDED**, **FEEDBACK_SENT**, **CANDIDATE_FEEDBACK_SENT**, **GRANT_CREATED**, **GRANT_VOTE**, **GRANT_QUEUED**, **GRANT_EXECUTED**, **GRANT_CANCELED**

Omit **type** to get all events mixed together.

Response:

```
{  
  "events": [  
    {  
      "type": "VOTE",  
      "blockNumber": 12345678,  
      "timestamp": "2026-03-13T...",  
      "txHash": "0x...",  
      "data": { "voter": "0x...", "proposalId": 123, "support": 1, "votes": 4, "reason": "..." }  
    }  
  ],  
  "hasMore": true,  
  "oldestBlock": 12345000  
}
```

Farcaster Feed

```
GET /api/feed/nouns    # /nouns channel  
GET /api/feed/noc      # /noc channel
```

Returns Farcaster casts from specified channels via Neynar API.

ENS Resolution

```
GET /api/ens?address=0x1234...
GET /api/ens?name=vitalik.eth
```

Noun Holders

```
GET /api/noun-holders
```

Returns Noun holder leaderboard with ENS names.

ASCII Art

```
GET /api/ascii-image?nounId=123
```

Generates ASCII art representation of a Noun.

Health Check

```
GET /api/health
```

Agent NounIRL

nounirl.eth is an autonomous onchain agent that settles Nouns auctions when specific traits appear. It runs inside the Ponder API process on Railway.

How It Works

1. Users tip ≥ 0.002 ETH to **nounirl.eth** on any supported chain (Ethereum, Base, Optimism, Arbitrum, Zora)
2. Specify desired traits: "shark head AND blue noggles"
3. Agent monitors every block and predicts the next Noun's traits using the NounsSeeder algorithm (keccak256 pseudorandom derived from block hash + noun ID)
4. When a match is found and the auction has ended, the agent calls **settle()** to mint the Noun with the predicted traits

Agent API Endpoints

Endpoint	Method	Description
/api/agent/status	GET	Running state, current block, predictions, wallet balance

Endpoint	Method	Description
<code>/api/agent/predict</code>	GET	Predicted traits for the next Noun
<code>/api/agent/reserve</code>	POST	Create trait reservation (requires tip tx hash)
<code>/api/agent/reservations</code>	GET	List reservations (<code>?wallet=0x...</code> to filter)
<code>/api/agent/cancel/:id</code>	POST	Cancel a reservation
<code>/api/agent/check</code>	POST	Force a manual block check
<code>/api/agent/traits/:category</code>	GET	List trait names (<code>head</code> , <code>body</code> , <code>accessory</code> , <code>glasses</code> , <code>background</code>)
<code>/api/agent/parse-traits</code>	POST	Parse natural language to structured trait filters
<code>/api/agent/settlements</code>	GET	History of successful settlements
<code>/api/agent/deploys</code>	GET	Autonomous deploy history
<code>/api/agent/deploy</code>	POST	Trigger autonomous code deploy (requires ≥ 4 Nouns)

Trait Categories

background: 2 options (cool, warm)
body: 31 options
accessory: 144 options
head: 258 options
glasses: 24 options

Use `/api/agent/traits/head` to list all head trait names, etc.

Small Grants Treasury

Contract: `0xBAc9233725440c595b19d975309CC98cb259253a` (Ethereum mainnet)

A noun.wtf-exclusive micro-governance system. Combined governor + treasury in one contract.

Parameters

Parameter	Value	Notes
Voting delay	0 blocks	Voting starts immediately
Voting period	3,600 blocks	~12 hours
Timelock delay	43,200 seconds	12 hours after vote passes
Grace period	604,800 seconds	7 days to execute after timelock
Max operations	10	Per proposal
Quorum	None	1 FOR / 0 AGAINST passes

Lifecycle

propose() → ACTIVE (12hr vote) → SUCCEEDED → queue() → QUEUED (12hr timelock) → execute()
 → DEFEATED (for <= against)

Key Functions

```
// Anyone can propose
propose(address[] targets, uint256[] values, string[] signatures, bytes[] calldatas, string description) → uint256

// Noun holders vote (voting power from NounsToken.getPriorVotes)
castVote(uint256 proposalId, uint8 support) // 0=against, 1=for, 2=abstain
castVoteWithReason(uint256 proposalId, uint8 support, string reason)

// After voting succeeds
queue(uint256 proposalId)

// After timelock passes
execute(uint256 proposalId)

// Proposer or admin can cancel
cancel(uint256 proposalId)

// Read state
state(uint256 proposalId) → ProposalState
proposals(uint256 proposalId) → (proposer, snapshotBlock, startBlock, endBlock, eta, forVotes, againstVotes, ab
getReceipt(uint256 proposalId, address voter) → (hasVoted, support, votes)
```

Funding

Send ETH directly to the contract address. The `receive()` function accepts any deposit and emits `ETHReceived(sender, amount)`.

Terminal Chat

The terminal ([/terminal](#) or new-mode homepage) provides an AI chat interface backed by Claude with governance tool access.

POST /api/chat

```
{
  "message": "vote for prop 948",
  "wallet": "0xae4705dc...",
  "history": [
    { "role": "user", "content": "previous message" },
    { "role": "assistant", "content": "previous response" }
  ]
}
```

Response when the agent wants to execute a governance action:

```
{
  "message": "prepared your vote...",
  "stop_reason": "end_turn",
  "pending_action": {
    "type": "GRANT_VOTE",
    "grantId": 1,
    "support": 1,
    "reason": "testing"
  }
}
```

Available Governance Actions

The terminal can prepare these actions for wallet signing:

Action Type	What It Does
VOTE	Vote on a DAO proposal (with gas refund via client ID 37)
PROPOSAL_FEEDBACK	Send signal on a proposal (no gas refund)
CANDIDATE_FEEDBACK	Send signal on a candidate
CREATE_CANDIDATE	Create a new proposal candidate
UPDATE_CANDIDATE	Update an existing candidate
SPONSOR	Sign an EIP-712 sponsor signature for a candidate
PROMOTE	Promote a candidate to a full onchain proposal
BID	Place a bid on the current Noun auction
GRANT_VOTE	Vote on a Small Grants proposal
GRANT_PROPOSAL	Create a new Small Grants proposal

Rate Limits

- 10 messages per minute per wallet/IP
 - 50 messages per day per wallet/IP
-

Local Development

Prerequisites

- Node.js >= 18
- pnpm 10.x (`corepack enable && corepack prepare pnpm@latest --activate`)
- PostgreSQL (for Ponder indexer) or use Railway
- An Infura API key (free tier works for development but slow)

Clone & Install

```
git clone https://github.com/user/noun-wtf.git
cd noun-wtf
pnpm install
```

Run the Frontend

```
cd packages/nouns-webapp

# Create .env
cat > .env << 'EOF'
VITE_CHAIN_ID=1
VITE_MAINNET_JSONRPC=https://ethereum-rpc.publicnode.com
VITE_MAINNET_SUBGRAPH=https://spirited-flexibility-production-3c30.up.railway.app
EOF

# Skip wagmi codegen in dev
CI=true pnpm dev
```

The app runs at <http://localhost:5173>. It will use the production Ponder API for data.

Run the Ponder Indexer Locally

```
cd packages/nouns-api

# Create .env.local
cat > .env.local << 'EOF'
PONDER_RPC_URL_1=https://mainnet.infura.io/v3/YOUR_INFURA_KEY
DATABASE_URL=postgres://user:pass@localhost:5432/nouns_ponder
EOF

pnpm dev
```

Ponder starts syncing from the configured start blocks. At 2 RPS (free Infura), a full sync from block 12,985,438 takes ~8-12 hours. The GraphQL API is available immediately at <http://localhost:42069/graphql> with partial data while syncing.

Important: Do NOT add free public RPCs (publicnode, llamarpc) to Ponder config. The Stream factory creates 38+ addresses and free RPCs reject multi-address `eth_getLogs` queries, crashing the indexer.

Run on Sepolia Testnet

Set `PONDER_CHAIN=sepolia` in your environment. The config will use Sepolia contract addresses and a Sepolia RPC automatically.

Build for Production

```
# Frontend
cd packages/nouns-webapp
CI=true pnpm build    # outputs to dist/

# Full monorepo
cd /path/to/noun-wtf
pnpm build            # turbo builds all packages
```

Environment Variables

Frontend (`packages/nouns-webapp/.env`)

Variable	Required	Description
<code>VITE_CHAIN_ID</code>	Yes	<code>1</code> for mainnet, <code>11155111</code> for Sepolia
<code>VITE_MAINNET_JSONRPC</code>	Yes	Ethereum RPC URL (publicnode works for frontend reads)
<code>VITE_MAINNET_SUBGRAPH</code>	Yes	Ponder API URL
<code>VITE_WALLET_CONNECT_V2_PROJECT_ID</code>	No	WalletConnect project ID (mobile wallet support)
<code>VITE_ETHERSCAN_API_KEY</code>	No	For wagmi codegen (skipped in CI)

Ponder API (`packages/nouns-api/.env.local`)

→ EXPIRED (grace period passed without execution)

Trait Indices

Noun traits are stored as integer indices. To resolve them to names, use the [@noundry/nouns-assets](#) package or query the NounsDescriptorV2 contract:

```
background: 0=cool (grey), 1=warm (beige)
body:       0-30
accessory:  0-143
head:       0-257
glasses:    0-23
```

The `/api/agent/traits/:category` endpoint returns human-readable names for each index.

Ponder GraphQL Patterns

All plural queries return `{ items: [...] }` (Ponder v0.12+ convention):

```
# CORRECT
{ proposals { items { id } } }

# WRONG — this doesn't work
{ proposals { id } }
```

Single item queries use the table name (singular):

```
{ proposal(id: "123") { id status description } }
```

Build Ideas

Here are concrete things you could build on this stack in a hackathon:

Data / Analytics

- **Voter influence graph** — Use the delegates + votes data to map who influences what
- **Client ID leaderboard** — Track which frontends facilitate the most governance activity
- **Proposal outcome predictor** — Use historical vote patterns to predict active proposal outcomes
- **Treasury flow visualization** — Streams + proposal execution data shows where money goes

Governance Tools

- **Delegation marketplace** — Help Noun holders find aligned delegates using vote history
- **Proposal diff viewer** — Track ProposalUpdated events to show what changed between versions
- **Multi-sig governance** — Build a UI for coordinated voting across multiple Noun holders
- **Grant proposal templates** — Pre-built SmallGrantsTreasury proposals for common requests

Agent Extensions

- **Custom settlement strategies** — Fork the trait predictor for different auction strategies
- **Cross-DAO agent** — Extend nounirl to watch Lil Nouns, Purple DAO, etc.
- **Farcaster bot** — Post settlement predictions + governance activity to Farcaster
- **Telegram/Discord bridge** — Pipe the terminal chat into group chats

Frontend

- **Mobile-first governance** — The current terminal works on mobile but a native-feeling app would be better
- **Proposal builder** — Visual drag-and-drop for constructing proposal transactions
- **Noun trait explorer** — 3D viewer for all possible trait combinations with rarity scores
- **ENS-gated communities** — Use Noun ownership data for access control

Onchain

- **Extend SmallGrantsTreasury** — Add features like recurring grants, milestone-based payouts, or quadratic voting
- **Proposal sponsorship market** — Candidates need sponsor signatures; build a market for matching proposers with sponsors
- **Noun fractionalization** — Split Noun voting power across multiple wallets

Links

Resource	URL
noun.wtf (live)	https://noun.wtf
GraphQL API	https://spirited-flexibility-production-3c30.up.railway.app/graphql
SmallGrantsTreasury (Etherscan)	https://etherscan.io/address/0xBAC9233725440c595b19d975309CC98cb259253a
NounsToken (Etherscan)	https://etherscan.io/address/0x9C8fF314C9Bc7F6e59A9d9225Fb22946427eDC03
NounsDAOV4 (Etherscan)	https://etherscan.io/address/0x6f3E6272A167e8AcCb32072d08E0957F9c79223d
Nouns Center (ecosystem docs)	https://nouns.center

Resource	URL
Nouns Protocol (official)	https://nouns.wtf